CPSC 2720 Winter 2026

Text Adventure Game

Group 16

Curt Davison, Austin Froese, Aodhan Bevans

4/7/16

Introduction -

This project involved the design and implementation of a text-based adventure game. Our game was built around the core ideas of exploration, interaction, and survival. The player moves through a world made up of multiple locations, interacts with non-player characters, uses items, solves puzzles, and works toward collecting a game winning item. This report explains how the project was managed, how the software design was developed, and how the final implementation satisfies the requirements of the assignment. It also documents the team's organization, development process, risk management decisions, and design rationale, following the structure provided in the project report template.

Project Management

Team Roles

Team Member	**Design - Draft**	**Design – Final**	**Implementation - Basic**	**Implementation - Final**
Curt	Phase Lead	Design Lead	Reporting lead	Reporting Lead/QA lead
Austin	Design Lead	QA Lead	Phase lead	Phase Lead
Aodhan	QA Lead	Reporting Lead	Design lead	Design Lead/QA lead

Team Role Responsibilities -

Throughout the project phases we rotated roles to ensure proper coverage and ensure equal and fair work for the group. We accomplished this through communication on what needs to be done.

Phase Lead
The Phase Lead helped keep the group on track and made sure work got done on time.

Design Lead
The Design Lead helped with planning the game and the class design.

QA Lead
The QA Lead helped with testing, finding bugs, and checking that the code worked.

Reporting Lead

The Reporting Lead helped write the report and organize the project information.

Development Process

Risk Management -

One risk was falling behind on work, so we tried to split tasks clearly. This was also reinforced by our group support, making sure we kept each other on track. Another risk was developing isolated code due to the switching of roles. We made sure to refer to each other or reference materials and often test code together to avoid this issue. We also had to watch for hidden bugs, crashes, and memory problems that came with progress or mixing push/pulls. Proper communication between members and general code management was key in helping lower these risks.

Code Review Process-

We checked each other's code before using it in the project, giving feedback where needed. As roles shifted throughout development, we would refer to other members to ensure the code was continuous and flowed naturally together. We looked for bugs, style problems, and whether the code matched the design, making sure to adapt where necessary. We also made sure the code passed foundational tests throughout development. This led to two major testing periods near the middle and end of the project as status and quality check points. This allowed us to gather more data to catch any bugs or mistakes early on and helped actually form what the current project looks like.

Communication Tools -

We made sure to use accurate commit messages to communicate what we worked on. If any of us were unsure of current implementations, future direction, design problems, how to tackle bugs, or had questions in general, we were able to easily reach another. This allowed for better universal understanding and team unison as the project unfolded. In tandem, we also held mini strategy meetings and progress reports about twice a week in person. This ensured everyone was a part of the project's development and guaranteed a working model on all fronts.

Changing Management -

Sometimes we had to change parts of the project while we were working. While the reference material was used to stay on track, we kept ourselves and the development flexible. This worked well with our ever-changing roles as it kept ideas fresh and implementations malleable. In addition, being uncommitted to a single role allows each member to act as a debugger, reviewer, or analyzer for another. If the current design of a class or function (or overall approach to an issue) seems off, we would talk together to find out why and figure out a better way to move forward. From there on, we were able to change and adapt the code, and even design, for better as needed.

Software Design

Design – Class diagrams

Design – Sequence diagrams

Class Descriptions -

Game
The Game class controls the main parts of the game. It helps run the game, checks if the player wins or loses, and handles the game flow.

Player
The Player class stores the player's health, hunger, attack, weapon, inventory, and current location.

WorldMap
The WorldMap class stores the game map and helps move the player from one location to another.

Location
The Location class is the base class for places in the game. It can hold NPCs, mobs, items, and recipes.

Terrain / Village / Unique
These classes are different kinds of locations in the game world.

Inventory
The Inventory class stores items and lets the game add, remove, and check for items.

Item
The Item class is the base class for all items in the game.

Material
The Material class is used for basic items like resources.

Food
The Food class is used for items the player can eat to change hunger.

Weapon

The Weapon class is used for items that can improve attack power.

NPC
The NPC class is the base class for non-player characters.

HelpNPC
The HelpNPC class gives hints or help to the player.

ShopNPC
The ShopNPC class lets the player trade items.

Mob
The Mob class represents enemies in the game.

CraftingRecipe
The CraftingRecipe class stores what items are needed to make or trade for another item.

ScreenDisplay
The ScreenDisplay class prints game information to the screen.

UserInput

The UserInput class handles player input and checks that it is valid.

Appendices

Appendix A: Figures and Tables